

Ride Alert

Q&A: “The Power of Friendship”

Serj Artuni Erik Gonzalez Tewodros Asres Amin Zoghلامي
Hayk Mirzahanyan

Raw Notes / Draft

How they break up the workload within the team. Is there someone in charge of certain aspects of the team? Coding, presenting, etc...

Our system is comprised of 5 main components:

- arduino
- speaker
- bluetooth module
- gyro/sensor
- phone

to us, the most logical sense was to assign each of us one component to focus on and primarily develop for

- Serj Artuni: as everyone works on their module, serj is continually working with our changing code to make sure each component interfaces correctly outside of the simulator
- Erik Gonzalez: primarily working on the speaker programming the logic to activate it based on data from the gyro sensor
- Tewodros Asres: programming and interfacing with the gyro/tilt sensor to be able to accurately read data from the sensor
- Amin Zoghلامي: programming the bluetooth module to accurately send alerts to the users phone device
- Hayk Mirzahanyan: programming the phone to receive data from the bluetooth module, and accurately send the correct alerts

How have they been working on their documentations? Have you split the work across every member of the group or have you made it so only a single individual manages that?

We have shared docs, each focused on different parts of our project. For example, we have a form where we document the current bugs and errors, and keep record of past bugs and their fixes.

We felt it was necessary that we all bear the same load of work in all aspects of our project. So, for each of our respective module we have a shared doc where the team member continually updates documenting any bugs, the fixes applied, etc. This way the work load is distributed evenly, while detailing the whole process from start to finish.

What tools are they using? How do they communicate and collaborate?

The main tool we use is a combination of physical hardware, and virtual hardware. Due to only have a single arduino, bluetooth sensor, gyro, and etc., combined with the very small moments all our schedules align, we were forced to find a virtual solution. That led us to wokwi, an online website where you're able to play around with a virtual arduino (and modules such as bluetooth and gyro), you can wire the arduino virtually, pin to pin, to pretty much any arduino module in existence.

We decided on this platform due to it being the only one with a VS Code extension. So instead of having to consistently pass around a physical arduino, we can program in a virtual environment and when we are ready to test, we have a meeting to discuss any changes, explanations on how code works etc. This is where serj gets any updated code, or changes that clarifying so that he can test it physically.

What are some of the challenges that they are having and how are they getting around it? Is there a plan or just brut-force?

main challenges + how we get around :

- figuring out how to use wokwi vs code extension + how to use with version control
 - helping each other understand the extension and sharing what we learned on our own time
- sending and reading data to a phone via bluetooth. Garbled data, means nothing, sometimes will break or send garble halfway, wont send
 - reading documentations, seeing example code of how someone correctly sends and reads data from an android/ipone to an arduino
- reading data from gyro, correctly initializing and sending the correct I/O instructions
 - reading gyro docs helped, excellent tutorials on how to get a gyro sensor to send data

- sometimes our code that works in the sim will require tweaking to work on a real arduino.
 - yea this one is just brute force until it works really
 - scheduling, meetings are as consistent as we would like
 - we substitute with zoom
-

Responses

How we divide the workload within the team

Our project, *Ride Alert*, is built around five main components: the Arduino, speaker, Bluetooth module, gyro sensor, and phone. So it just made sense for each of us to program a component. Serj focuses on integration, working with our shared code to make sure all the modules talk to each other properly outside of the simulator. Erik handles the speaker logic, making sure it activates when it's supposed to based on the gyro's data. Tewodros programs the gyro sensor so we get accurate data. Amin programs the Bluetooth module, making sure it sends the correct alerts to the phone, and Hayk handles the phone code, making sure it receives data correctly and sends the correct alert.

How we handle documentation

We agreed that everyone should share the documentation workload, like our components. Each of us maintains a doc for our module where we track bugs, fixes, test results, and updates. This way, all of us carry the same workload in programming and documentation. It helps us stay organized, while also being updated on everyone's work.

Tools and collaboration methods

Because we only have one physical Arduino setup and conflicts in our schedules, we use both real and virtual tools to collaborate. Our main platform is Wokwi, which lets us simulate Arduino hardware online, from the Bluetooth module to the gyro. It also integrates with VS Code, so we can code, test, and share our updates easily without needing to pass the physical board around. Once code is ready for real-world testing, Serj handles that part. We meet over Zoom whenever we need to go over updates, explain new changes, or plan testing sessions.

Challenges and how we deal with them

We've definitely had our share of challenges. At first, figuring out how to use the Wokwi VS Code extension with version control took some trial and error, but we got through it by helping each other out and sharing what we learned. Sending and reading data over Bluetooth was tricky. Sometimes it would come through garbled or incomplete, so we dug through docs and example projects until we found solutions. The gyro sensor also took a

while to get going and a lot of reading before we got it working consistently. Sometimes, code that works fine in the simulator just doesn't work on a real Arduino, and yeah, that's where just brute-force changes until we get it working. We just keep tweaking until it works. Lastly, scheduling has been tough, so we rely on Zoom meetings and shared docs to stay on track when we can't meet in person.